High Level Design (HLD)

E-Commerce Product Management System

Prepared from project repository analysis

Document Purpose

This HLD gives sprint evaluators a clear architecture-level view of the application: what has been built, how modules communicate, which responsibilities each layer owns, and how authentication, workflow, reporting, audit, and NUnit-based testing fit into the system.

Item	Description
Project Type	E-Commerce Product Management Platform
Frontend	Angular 21 single-page application
Backend	ASP.NET Core microservice-style APIs
Gateway	Ocelot API Gateway
Database	SQL Server with service-owned EF Core DbContexts
Messaging	RabbitMQ with MassTransit
Testing	NUnit unit and integration test projects

1. System Context

User / Business User

Accesses storefront, dashboards, catalog, workflow, reporting and audit features

↓

Angular Frontend

Role-based UI, route guards, API services, JWT interceptors, dashboards and workflows

↓

Ocelot API Gateway

Single secured backend entry point; routes requests to downstream services

↓

Backend Microservices

Identity.API | Catalog.API | Workflow.API | Reporting.API

↓

Persistence and Messaging

SQL Server databases per service + RabbitMQ event bus for reporting and audit updates

2. Logical Architecture

Layer	Component	Responsibility
Presentation Layer	Angular Frontend	Provides public storefront, login/register, role dashboards, product management, workflow, inventory, reports, audit and profile screens.
Gateway Layer	Gateway.API / Ocelot	Central route forwarding, JWT validation, CORS control, Swagger aggregation, logging and QoS policies.
Identity Service	Identity.API	Authentication, authorization data, users, roles, refresh tokens, password reset and profiles.
Catalog Service	Catalog.API	Products, categories, variants, media assets, SKU generation and catalog visibility.
Workflow Service	Workflow.API	Pricing, inventory, product review submission, approval status and pending approvals.
Reporting Service	Reporting.API	Dashboard KPIs, product reports, CSV export, historical snapshots and audit trails.
Messaging Layer	RabbitMQ / MassTransit	Asynchronous event delivery from business services to Reporting.API.
Data Layer	SQL Server / EF Core	Service-owned persistence using IdentityDBContext, CatalogDbContext, WorkflowDbContext and ReportingDbContext.

3. Backend Service Map

Service	Port	Main API Paths	Primary Data Owned
Identity.API	7283	/api/auth, /api/users	Users, RefreshTokens, PasswordResetTokens
Catalog.API	7029	/api/products, /api/categories	Products, Categories, ProductVariants, MediaAssets
Workflow.API	7245	/api/workflow	Prices, Inventories, Approvals
Reporting.API	7118	/api/reports, /api/audit	ProductReports, DashboardSnapshots, AuditLogs
Gateway.API	7292	Frontend-facing /api routes	No business data; gateway configuration only

4. API Gateway Routing

Frontend Path	Downstream Service	Access Pattern
/api/auth/*	Identity.API	Public for login/register/reset; authenticated for logout/profile
/api/users/*	Identity.API	Authenticated; Admin for user administration
/api/products/*	Catalog.API	GET public with visibility rules; writes require authorized roles
/api/categories/*	Catalog.API	GET public; writes require authorized roles
/api/workflow/*	Workflow.API	Authenticated workflow operations; selected read access for pricing
/api/reports/*	Reporting.API	Admin and ProductManager
/api/audit/*	Reporting.API	Admin only

5. Data Ownership

Bounded Context	DbContext	Tables / Entities	Purpose
Identity	IdentityDBContext	Users, RefreshTokens, PasswordResetTokens	Owns user identity, roles, account status and authentication token data.
Catalog	CatalogDbContext	Products, Categories, ProductVariants, MediaAssets	Owns product master data and catalog structure.

Workflow	WorkflowDbContext	Prices, Inventories, Approvals	Owns operational product workflow data separate from catalog master data.
Reporting	ReportingDbContext	ProductReports, DashboardSnapshots, AuditLogs	Owns read-optimized reporting and audit records built through events.

6. Event-Driven Communication

Identity.API / Catalog.API / Workflow.API

Publish domain events when users, products, pricing, inventory or approval status change

↓

RabbitMQ + MassTransit

Durable asynchronous messaging boundary between services

Reporting.API Consumers

Update product reports, dashboard counts and audit trail records

Event	Published By	Consumed By	Business Purpose
UserCountChangedEvent	Identity.API	Reporting.API	Maintains total user count in dashboard reporting.
ProductReportChangedEvent	Catalog.API	Reporting.API	Keeps product reporting read model synchronized with catalog changes.
ProductStatusChangedEvent	Workflow.API	Reporting.API	Records approval/status changes for reporting and audit visibility.
AuditLogCreatedEvent	Identity/Catalog/Workflow	Reporting.API	Persists audit trail entries for important business actions.

7. Security and Roles

Security is enforced through Angular route guards, HTTP interceptors, JWT bearer authentication at the gateway and downstream APIs, and role-based authorization on controllers.

Role	Allowed Capabilities
Customer	Browse public storefront, view approved products, manage own profile/account.
ContentExecutive	Create and maintain product content, submit products for review.
ProductManager	Manage products, pricing, inventory and view reporting dashboards.
Admin	Manage users, approve/reject workflow items, view reports, audit trail and administrative dashboards.

8. Key Functional Flows

Authentication Flow

Login Request

Angular calls /api/auth/login through Gateway.API

↓

Identity Validation

Identity.API validates email, password and active account status

↓

Token Issued

JWT access token and refresh token returned to frontend

Product Management Flow

Product Created or Updated

Catalog.API validates category and applies product business rules

↓

Catalog Saved

Product data is stored in CatalogDbContext

↓

Report Event Published

ProductReportChangedEvent updates Reporting.API read model

Approval Workflow Flow

Submit for Review

ContentExecutive/ProductManager/Admin submits product through Workflow.API

↓

Admin Decision

Admin approves or rejects pending approval

↓

Status Event and Audit

Workflow publishes status/audit events for Reporting.API

9. Frontend Module Overview

Frontend Area	Purpose
core	Authentication state, token storage, services, guards, interceptors and global error handling.
shared	Reusable UI components such as navigation, loading spinner, confirm dialog and validation messages.
auth	Login, register, forgot password and reset password screens.
storefront	Public home page, product browsing and product detail screens.
catalog	Product list, product form, categories, variants and media management.
workflow	Product workflow screen, approval screen, workflow badges and timeline components.
dashboard	Role-specific dashboards for Admin, ProductManager, ContentExecutive and Customer.
reporting	Dashboard KPIs, product reports and export flow.
audit	Admin audit trail viewer.
profile	View/edit profile and change password.

10. NUnit Testing Coverage Overview

The solution uses NUnit as the backend testing framework. Test projects exist for each major backend service and are organized around unit and integration test coverage. This supports sprint evaluation by showing that the system is not only implemented but also validated at service and API boundaries.

Test Project	Scope	Testing Tools
Catalog.API.Tests	Catalog services, repositories and controllers	NUnit, NUnit3TestAdapter, Moq, EF Core InMemory, ASP.NET Core MVC Testing
Identity.API.Tests	Authentication, user service, repositories and controllers	NUnit, NUnit3TestAdapter, NUnit.Analyzers, Moq, EF Core InMemory, coverlet.collector
Workflow.API.Tests	Workflow controllers and approval/pricing/inventory behavior	NUnit, NUnit3TestAdapter, NUnit.Analyzers, EF Core InMemory, coverlet.collector

Reporting.API.Tests	Reports controller and reporting behavior	NUnit, NUnit3TestAdapter, NUnit.Analyzers, EF Core InMemory, coverlet.collector

Testing Layer	Description
Unit Tests	Validate application service logic using NUnit test fixtures and Moq-based dependencies.
Repository Integration Tests	Validate repository behavior using EF Core InMemory test databases.
Controller Integration Tests	Use WebApplicationFactory-based test hosting to validate API endpoint behavior.
Coverage Support	coverlet.collector is configured in Identity, Workflow and Reporting test projects for coverage reporting.

11. Non-Functional Architecture Considerations

Concern	Design Support
Scalability	Services are separated by business capability and can be deployed/scaled independently in a future environment.
Security	JWT authentication, refresh tokens, gateway validation, downstream validation and role authorization.
Maintainability	Layered structure with controllers, application services, repositories, DTOs and domain entities.
Observability	Serilog request logging and service-level exception handling middleware.
Reliability	Ocelot QoS options with Polly and asynchronous RabbitMQ messaging for reporting/audit updates.
Testability	Dedicated NUnit test projects with unit and integration test organization.